

Manual: De modelo a agente en producción

Messages API → Claude Agent SDK → Claude Managed Agents

Fuente: video de X (11 min 52 s) — post de **Mo Elgaraihy** (@EngMoElgaraihy), 26 SEP 2026 **Link:** <https://x.com/engmoelgaraihy/status/2103658343512297936> **Charla:** "Agentic Engineering Summit" (AWS Builder Loft, con branding AWS) — lightning talk **Orador: Gagan Bhat**, Member of Technical Staff en **Anthropic** (según la placa de título del video: "Gagan Bhat — Member of Technical Staff | ANTHROPIC"; en el audio se presenta como "product engineer at Anthropic"). El post de X es de Mo Elgaraihy, que difundió la charla. **Procesado por:** Lango (yt-dlp → ffmpeg → Whisper small → OCR de slides con tesseract)

Nota de fidelidad: la transcripción automática confundía sistemáticamente "agent/agentic" con "Asian" y "Claude" con "Cloud". Este manual está **corregido**, con las correcciones dudosas marcadas como [?] . Al final hay un apéndice con la transcripción cruda y el detalle de correcciones.

1. Resumen ejecutivo (TL;DR)

La charla explica **cómo evolucionó la "superficie" que usa un dev para construir agentes** en Anthropic, en tres niveles acumulativos. Cada nivel te quita una capa de infraestructura de encima:

Nivel	Producto	Qué te da Anthropic	Qué seguís construyendo vos
1	Messages API	El modelo (tokens in / tokens out)	<i>Todo</i> : loop agéntico, contexto, tools, sesiones, auth, observabilidad
2	Claude Agent SDK	El harness agéntico (loop, tools built-in, sandbox, filesystem)	Hosting, escalado, estado de sesión, observabilidad, tools custom
3	Claude Managed Agents	El harness + runtime + sandbox + sesiones + auth + vault + hosting + observabilidad	El producto y la lógica de negocio

Tesis central: con Managed Agents pasás de "horas para el prototipo y semanas para producción" a **shipear rápido sin reescribir nada** — el mismo código de prototipo es *production-grade*.

Cambio arquitectónico clave: el **loop agéntico se separa del sandbox**. El "cerebro" (modelo + orquestación) vive en el cloud; las "manos" (tools, sandbox) se levantan **solo cuando se necesitan**. Antes, loop y ejecución de tools compartían contenedor → latencia alta en cada arranque y *single point of failure*.

2. La evolución de las superficies agénticas (3 slides)

Nivel 1 — 01 Messages API

Slide: "The model is provided. Everything around it is yours to build."

- Antes, Anthropic te daba **tokens in / tokens out**. Nada más.
- Diagrama del slide:
- **Your product → Agentic loop** (*calls Claude, runs tools, manages context*) → **Claude Messages API**
- **Production infrastructure** (todo *run by you*): session management · hosting · observability · credentials · infrastructure
- Vos manejabas: loop agéntico, manejo de contexto, ejecución de tools, estado de sesión y recovery, auth, observabilidad, lógica de tool custom.

Nivel 2 — 02 Claude Agent SDK

Slide: "The agentic harness is provided."

- Aparece **Claude Code** (la implementación de Anthropic de un *agentic harness*) y se empaqueta como **Claude Agent SDK**.
- Ahora el SDK te provee: **el loop agéntico + el harness**, manejo de contexto, ejecución de tools built-in, **sandbox** y **filesystem**.
- Por debajo sigue usando Messages API (tokens in / out).
- **Vos seguís manejando:** hosting y escalado, estado de sesión, observabilidad, lógica de tools custom. → *Todavía hay mucho trabajo para llevar el agente a producción.*

Nivel 3 — 03 Claude Managed Agents

Slide: "You own the product. Agents are production ready."

- Vos traés **solo**: tu *task* + la **config del agente** + tu **lógica de tools custom** (si hace falta).
- Anthropic maneja: harness purpose-built · runtime de tools + **sandbox** · estado de sesión, persistencia y **checkpointing** · auth y **credential vaults** (guardado seguro de API keys de tus otros servicios) · hosting, escalado y observabilidad.
- Por debajo, sigue siendo tokens in / tokens out.
- **Slide 03 — arquitectura final:**
- **Your product → Agentic loop** (*caching, compaction, and tools built in*) → **Claude Messages API**
- **Sandbox**: contenedor seguro con **filesystem persistente**
- **Credentials**: auth y secrets gestionados
- **Production infrastructure**: session management · hosting · observability
- **Recomendación explícita del orador**: si querés ir de *idea* a un agente feature-rich en tu producto, **empezá por Managed Agents**.

3. El cambio arquitectónico: cerebro separado de las manos

(~05:26 en el video)

Antes: - El loop agéntico y la ejecución de tools corrían **en el mismo contenedor** que hosteaba al agente. - Costo: **latencia alta** cada vez que arrancabas un agente + **single point of failure** (si caía el contenedor, caía el loop).

Con Managed Agents: - El **agentic loop corre server-side** (en el cloud), **separado del sandbox**. - Analogía del orador: "*el cerebro se separa de las manos*" — modelo / orquestación por un lado; tools y sandbox por el otro. - **Por qué importa**: no todo caso de uso necesita sandbox → **se levanta solo cuando se necesita**. Eso permite escalar mucho más rápido y da [?] **menor latencia y mayor disponibilidad**. - **Consecuencia operativa**: podés **cerrar la laptop** y el agente sigue corriendo en el cloud. Vos *dirigís* el loop (mandás eventos, lo empujás en distintas direcciones), no lo hosteás.

4. Los 3 recursos primarios de Managed Agents

(~03:53)

4.1 Agent config — "*persona y capacidades*"

- Define **qué es tu agente**: modelo, prompts, tools, **MCP servers**, skills.
- **Versionado e inmutable** → podés iterar y **hacer rollback** si algo sale mal.

4.2 Environment — "*infraestructura y guardrails*"

- Define **en qué contenedor vive el agente**: frontera de red (networking boundary), **allowed hosts** y lo no permitido, acceso a internet, etc.
- Se persiste en el cloud → devuelve un **ID único**.
- Acá **montás los datos**: logs para que investigue, skills, archivos. Todo persistente en el cloud.

4.3 Sessions

- Tomás el **agent config** + el **environment** → **levantás una sesión**.
- Las sesiones **hablan en eventos**: el cloud te emite eventos de lo que el agente hace y vos le mandás eventos de vuelta para controlarlo/dirigirlo.
- La **conversación vive en el cloud** → las sesiones quedan **persistidas**. Multi-sesión out of the box; listás sesiones con un comando. Te olvidás de persistir/leer transcripts.

Hay más APIs ricas alrededor, pero **estos 3 recursos son el core** para configurar y shipear rápido.

5. Walkthrough: shipear tu primer managed agent (ejemplo "SRE agent")

(~06:40) El orador da el ejemplo de un agente SRE que investiga por qué un servicio de checkout está por encima del baseline.

Flujo en 4 pasos:

- 1. **Definís qué es el agente** — un bloque chico de código que describe tools, modelo y nombre. Al usarlo, **se persiste en el cloud**.
- 2. **Definís el contenedor/environment** — networking boundaries, acceso a internet, etc. Se persiste y te devuelve un **ID único**.
- 3. **Montás datos** — subís logs o skills que el agente va a usar durante la investigación. Persistente en el cloud.
- 4. **Arrancás la sesión** — usando el **agent ID + environment ID**. El agente en el cloud tiene un **sandbox real** con capacidades `grep / glob` para encontrar la root cause.

Punto clave: "todo esto, mientras escribís el código, es production-grade". Cuando estás conforme, usás **el mismo agent ID y environment** y lo metés en tu producto. **La escalabilidad viene built-in** → servir a tus usuarios requiere *cambios mínimos*.

Detalle de control: - Desde el cloud recibís **eventos**; si hay un **custom tool use** con necesidad local, lo manejas vos; si es un **MCP server**, éste se contacta directo. - Vos solo te ocupás de darle **el contexto correcto y la task correcta**. Del hosting del agente no te ocupás.

6. Features avanzadas (más allá de lo básico)

(~09:56)

- **Sub-agents / multi-agents** — podés levantar agentes en **jerarquía**.
- **Memory** — el agente aprende de sus sesiones y recuerda cosas en el tiempo → mejora solo.
- **Dreaming** — aprende de los transcripts a lo largo del tiempo, **implementa skills** y **mejora el memory store** para rendir mejor en el futuro.
- **Outcomes** — itera sobre un producto varias veces (estilo "Ralph" loop) para dar un output de alta calidad.
- **Vault** — guarda tus **credenciales de forma altamente segura** (crítico).
- **MCP servers, webhooks, permission policies, interrupts**.
- **Console agent builder** — en la consola de Anthropic describís en texto plano qué agente querés y obtenés una **config production-grade** usable en tu producto.
- **Scheduled & triggered agents** — agents que corren como **cron job**: ej. un daily digest todos los días a las 8 a.m., o algo a la 1 a.m.

7. Mental model — los 7 takeaways

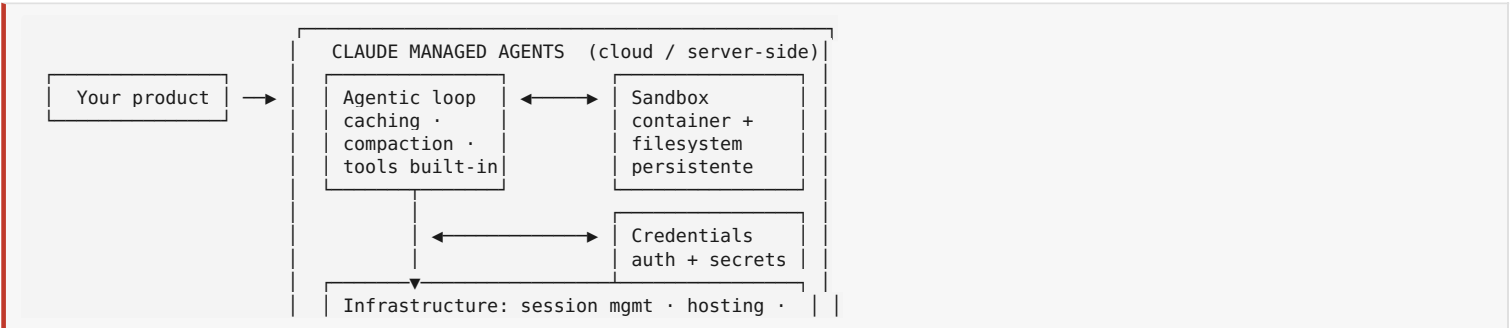
- 1. La evolución es **acumulativa**: cada capa te quita infraestructura, no te la agrega.
- 2. **Messages API** = el modelo. **Agent SDK** = el harness. **Managed Agents** = el producto llave en mano.
- 3. El **loop está server-side** → cerrás la laptop y sigue andando.
- 4. **Cerebro ≠ manos**: separar orquestación de sandbox = menos latencia, más escala, sin single point of failure.
- 5. **Sandbox on demand**: no todo caso de uso lo necesita.
- 6. **Agent config versionado e immutable** → iteración con rollback gratis.
- 7. **Prototipo = producción**: el mismo código que usás para prototipar se shipea.

8. Recursos mencionados

- **Repo:** GitHub anthropics → organización Anthropic → carpeta `cwc-workshops` (del evento **Code with Claude**).
- Adentro hay una carpeta que muestra **cómo shipear tu primer managed agent** (el ejemplo del SRE agent).
- **Console agent builder:** en la consola de Anthropic.
- **Contacto del orador:** email / LinkedIn; está en Anthropic y acepta feature requests.

△ El orador aclara: **parte de las APIs pueden cambiar** — chequear la doc del managed API al implementar. Además, ya **no está en beta** (ignorar referencias viejas a beta en el código de ejemplo).

9. Diagrama de la arquitectura (reconstruido de los slides)





Progresión de las 3 slides:

01 Messages API	02 Agent SDK	03 Managed Agents
----- [Your product] [Agentic loop]* [Messages API] [Prod infra]* (run by you)	----- [Your product] [Agentic loop] + Filesystem [Messages API] [Sandbox] [Prod infra] (run by you)	----- [Your product] [Agentic loop] ← provisto + caching/compaction [Messages API] [Sandbox + Credentials] [Prod infra] (run by Anthropic)

* = todo lo que seguís construyendo vos en ese nivel.

10. Apéndice A — Correcciones aplicadas a la transcripción

Whisper (modelo `small`, CPU) cometió errores sistemáticos. Principales:

Transcripción cruda	Corrección	Confianza
Asian / Asians	agent, agentic / agents	alta (contexto)
Cloud	Claude	alta
Cloud Managed Asians	Claude Managed Agents	alta
Cloud Asian SDK	Claude Agent SDK	alta
Cloud Code	Claude Code	alta
agentic furnace	agentic harness	alta
MCT servers	MCP servers	alta
checkwining	checkpointing	alta
Waltz	Vault	alta
Ralph looking move	"Ralph" loop (iteración)	media [?]
entropic / Anthropik	Anthropic	alta
"much more latency and higher liability"	"lower latency and higher availability"	media [?]
CWC workshops	Code with Claude workshops	alta
grab, glob capabilities	grep, glob capabilities	alta

11. Apéndice B — Transcripción corregida (integral)

[00:00]	Soy product engineer en Anthropic. Trabajo en cosas muy variadas: mejorar los modelos en tareas específicas, construir harnesses agénticos, trabajar con nuestros clientes más grandes para implementar casos de uso agénticos. Hoy vengo a hablar de cómo vimos la evolución de las superficies agénticas –las interfaces de construcción de agentes a las que la gente está acostumbrada– y cómo culminó en lo que construimos hace poco: Claude Managed Agents.
[00:43]	Cubrimos conceptos core: cómo funciona Managed Agents. Hay una parte que es un workshop hands-on, pero hoy no lo hacemos (no hay tiempo); cubro cómo sería. Y después hablamos de ir más allá de lo básico.
[00:59]	Al principio, en el amanecer de los modelos de lenguaje frontier, teníamos la Messages API : solo tokens in, tokens out. Eso es Claude. Le das el input y obtenés la predicción del próximo token. Eso era Messages API.
[01:25]	A medida que las cosas se complicaron y tuviste que construir agentes, loops y workflows más complejos, seguía existiendo la necesidad de manejar y construir tu propio loop agéntico , tu contexto, la ejecución de tools, el estado de sesión y recovery, auth, observabilidad, lógica de tool custom. Todo esto seguía siendo tuyo. Seguía siendo un problema.
[01:53]	Después vimos el rise de Claude Code , nuestra versión de un harness agéntico, y lo empaquetamos en el Claude Agent SDK . Este SDK provee el loop agéntico y el harness, manejo de contexto, ejecución de tools built-in del sandbox y filesystem. Por debajo sigue usando Messages API. Pero seguías teniendo que manejar hosting y escalado, estado de sesión, observabilidad, lógica de tools custom, etc. Todavía había mucho trabajo para llevar el agente a producción.
[02:29]	La tercera iteración de la evolución de los servicios agénticos es Claude Managed Agents . La idea: vos venís solo con tu task y la config de tu agente, y con tu lógica de

tools custom. Anthropic maneja: el harness purpose-built, el runtime de tools y el sandbox, el estado de sesión, persistencia y checkpointing, auth, credential vaults y guardado seguro de API keys para tus otros servicios. Y por supuesto hosting, escalado y observabilidad. No te tenés que preocupar por eso. Por debajo, tokens in / tokens out.

- [03:07] Así vimos la evolución de las interfaces que usan los devs para construir agentes. Managed Agents es nuestra forma más rápida de construir y shipear productos agénticos transformadores.
- [03:23] Si querés ir de la idea a tener un agente feature-rich en tu producto, te recomiendo empezar por Managed Agents. Les muestro cómo arrancar rápido y lo rápido que se lleva a producción. Vimos que los clientes tardan horas en llegar al prototipo y **semanas** en shipear a producción, por todo lo que hay que manejar alrededor del producto.
- [03:53] Hay **tres recursos primarios** en Managed Agents. Primero, **persona y capacidades**: qué es tu agente, qué modelo usa, los prompts, tools, MCP servers, skills. Todo esto está versionado e inmutable, así que podés iterar y hacer rollback. Segundo, **infraestructura y guardrails**: en qué contenedor vive el agente, cómo se define su frontera de red, cuáles son los allowed hosts, qué no está permitido. El environment donde vive el agente. Tercero, el recurso de **sessions**: tomás la config del agente, tomás el environment que configuraste, y levantás la sesión. Tres recursos simples.
- [04:55] Recordá: con Managed Agents, el **loop agéntico corre server-side**. La orquestación y ejecución del agente pasa en el cloud. Vos dirigís ese loop. Tus eventos entran y empujás el agente en distintas direcciones. Podés cerrar la laptop y el agente sigue en el cloud.
- [05:26] Un cambio arquitectónico clave: antes el loop agéntico y la ejecución de tools pasaban en **el mismo contenedor** donde se hosteaba el agente. Eso implicaba alto costo de latencia cada vez que arrancabas un agente, y un **single point of failure**: si caía el contenedor, caía tu loop. Con Managed Agents, el loop se **separa del sandbox**. El cerebro se separa de las manos, de las tools, del sandbox. Porque no todo caso de uso necesita un sandbox, y se puede levantar **solo cuando se necesita**. Es una diferencia crítica: permite escalar mucho más rápido y da `[?]` menor latencia y mayor disponibilidad.
- [06:17] Hay un workshop hands-on, pero no tenemos tiempo. Di esta charla en **Code with Claude** (nuestra conferencia de devs). Hay un **repo open source** en GitHub, organización Anthropic, carpeta **cwc-workshops**. Adentro hay una carpeta que muestra cómo shipear tu primer managed agent. Es un **SRE agent**.
- [06:40] Supongamos que tenés un incidente: tu servicio de checkout está por encima del baseline. Estaría bueno tener un SRE agent con un set rico de capacidades que investigue y encuentre qué pasa. El workshop muestra cómo implementarlo. Es simple.
- [07:00] Primero definís qué es tu agente: escribís un pedacito de código que describe las tools del agente, el modelo y el nombre. Al usarlo, se persiste en el cloud. Después definís dónde está tu contenedor: las fronteras de red, cuánto acceso a internet tiene, etc. Se persiste en el cloud y te da un ID único. Después **montás los datos**: si querés subir un log file para que el agente lo use al investigar, lo subís. Si querés subir skills, también. Todo persistente en el cloud. Y después usás el agente y el environment y **arrancás la sesión** (agent ID + environment ID).
- [07:56] El agente en el cloud tiene un **sandbox real** que puede usar para hacer capacidades grep/glob y encontrar la root cause del problema.
- [08:10] Todo esto, mientras escribís el código, es **production-grade**. Cuando estás conforme con tu agente, usás el mismo agent ID y environment, lo metés en tu producto y lo shipeás como agente real en tu plataforma. La escalabilidad viene built-in. Cuando esté listo el prototipo, lo servís a tus usuarios con cambios mínimos.
- [08:33] Recordá: las **sessions** hablan en eventos. Algunas cosas pueden cambiar, así que chequeá al usar el managed API. La idea: desde el cloud recibís eventos de lo que el agente hace, y mandás eventos de vuelta para controlarlo y mover su dirección.
- [08:54] Para ilustrar: el cloud agent corre en el cloud. Recibís cualquier **custom tool use** si hay necesidades locales, o el **MCP server** se contacta directo, y manejas y orquestás el agente desde tu producto. No te preocupás por hostear el agente. Solo te preocupás por darle el contexto correcto y la task correcta.
- [09:21] (Snippet de código que abre los streams y manda mensajes – no necesariamente relevante.)
- [09:28] Recordá: la conversación vive en el cloud. Parte de este código ya no está en beta, así que ignoralo. La conversación vive en el cloud; las sesiones quedan persistidas. Podés crear persistencia multi-sesión y listar todas tus sesiones en el cloud con un comando. No te preocupás por persistir un transcript, guardarlo, leerlo, etc.
- [09:56] Y hay muchas más features: **sub-agents / multi-agents** (agentes en jerarquía); **memory** (aprender de las sesiones y recordar en el tiempo, el agente mejora); **dreaming** (aprende de transcripts en el tiempo, implementa skills y mejora el memory store para rendir mejor a futuro); **outcomes** (itera sobre un producto varias veces, tipo "Ralph" loop, para dar un output de alta calidad); **Vault** (súper importante: guardar tus credenciales de forma altamente segura); **MCP servers**, **webhooks**, **permission policies**, **interrupts**. Hay incluso un **console agent builder** donde describís en texto plano qué agente querés y obtenés una config production-grade para tu producto. Y **scheduled / triggered agents**: que corran automáticamente, ej. a la 1 a.m. todos los días, o un daily digest cada mañana a las 8 a.m. – agents como cron jobs.
- [11:12] Con eso espero que tengan el mental model de managed agents y la evolución. El link está para empezar. Hay muchas más features, es muy feature-rich. Cualquier pregunta, email o LinkedIn. Estoy en Anthropic y podemos ver feature requests. Gracias.